
OPC-UA-Live-Dashboard

Echtzeitvisualisierung von Steuerungs- und Sensordaten

Projekt: Smart Home – System Integration

Autoren:

Stand: 17. Juni 2026

Dokumenttyp: System- und Benutzerdokumentation

Kurzbeschreibung: Das Dashboard liest Prozesswerte über OPC UA ein und stellt sie in einer Weboberfläche live dar. Dadurch können digitale Eingänge, Sensorwerte und Zustandsänderungen zentral beobachtet und analysiert werden.

Inhaltsverzeichnis

1	Zweck der Anwendung	3
1.1	Wofür wird das System verwendet?	3
1.2	Warum wurde das System entwickelt?	3
2	Systemübersicht	3
2.1	Aufgaben der Komponenten	4
3	Technischer Ablauf	4
3.1	Erfassung der Werte	4
3.2	Übertragung zum Browser	4
3.3	Ablauf in acht Schritten	5
4	Weboberfläche	5
4.1	Verbindungsstatus	5
4.2	Sensorauswahl	5
4.3	Widgets und Detailansicht	5
5	Inbetriebnahme	6
5.1	Voraussetzungen	6
5.2	Variante A: Betrieb mit Simulator	6
5.3	Variante B: Verbindung mit einer realen Steuerung	6
6	Fehlerbehebung	7
7	Grenzen und Sicherheit	7
8	Zusammenfassung	7

1 Zweck der Anwendung

1.1 Wofür wird das System verwendet?

Das OPC-UA-Live-Dashboard dient zur **Überwachung und Visualisierung von Prozesswerten**. Im Projekt werden damit beispielsweise digitale Eingänge eines Steckbretts, Temperaturwerte, Druckwerte oder Gerätezustände dargestellt. Die Bedienperson sieht im Browser sowohl den aktuellen Wert als auch dessen zeitlichen Verlauf.

Typische Anwendungsfälle sind:

- Kontrolle, ob ein Taster oder digitaler Eingang aktiv ist,
- Beobachtung von Sensorwerten während eines Versuchs,
- schnelle Fehlersuche bei einer Steuerung,
- Demonstration von OPC-UA-Kommunikation im Unterricht,
- Test des Dashboards mit simulierten Daten ohne reale Hardware.

1.2 Warum wurde das System entwickelt?

Werte einer Steuerung sind ohne zusätzliche Software meist nur direkt im Entwicklungswerkzeug oder an der Hardware sichtbar. Das Dashboard macht diese Werte über eine übersichtliche Weboberfläche zugänglich. Dadurch muss für eine einfache Kontrolle nicht jedes Mal die Steuerungssoftware geöffnet werden.

Die Lösung bietet insbesondere folgende Vorteile:

- **Zentrale Übersicht:** Mehrere Werte werden gemeinsam angezeigt.
- **Live-Aktualisierung:** Änderungen erscheinen ohne Neuladen der Webseite.
- **Plattformunabhängige Anzeige:** Ein moderner Browser genügt.
- **Flexible Datenquelle:** Reale Steuerung und Simulator werden unterstützt.
- **Erweiterbarkeit:** Weitere OPC-UA-Knoten können ergänzt werden.

Der Begriff **Live-Dashboard** ist hier genauer als “Web-Tracking”. Mit Tracking ist in diesem Projekt nicht die Analyse von Webseitenbesuchern gemeint, sondern das fortlaufende Beobachten technischer Prozesswerte.

2 Systemübersicht

Das System besteht aus vier Bereichen: Datenquelle, OPC-UA-Server, Backend und Weboberfläche. Der Browser verbindet sich nicht direkt mit OPC UA. Das Python-Backend übernimmt die Verbindung zur Steuerung und übersetzt die Daten in ein für die Webanwendung geeignetes Format.

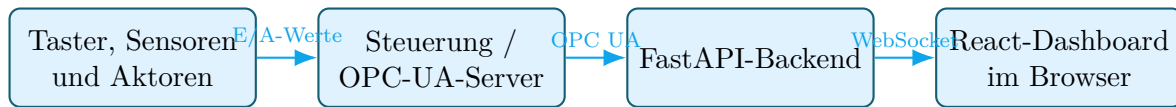


Abbildung 1: Vereinfachter Kommunikationsweg

2.1 Aufgaben der Komponenten

Komponente	Aufgabe
Taster und Sensoren	Erzeugen digitale oder analoge Eingangswerte.
Steuerung / Simulator	Stellt Variablen über einen OPC-UA-Endpunkt bereit.
FastAPI-Backend	Verbindet sich als OPC-UA-Client, abonniert passende Knoten und sammelt deren Werte.
WebSocket	Überträgt die aktuellen Daten fortlaufend vom Backend an alle verbundenen Browser.
React-Frontend	Stellt Werte, Status, Einschaltdauer und einen kurzen Verlauf als Widgets dar.

3 Technischer Ablauf

3.1 Erfassung der Werte

Ein Taster sendet nicht selbstständig Daten über OPC UA. Stattdessen liest die Steuerung den elektrischen Zustand des Tasters ein. Die zugehörige Variable wird anschließend vom OPC-UA-Server der Steuerung bereitgestellt.

Das Backend durchsucht den Objektbaum des OPC-UA-Servers. Knoten mit passenden Bezeichnungen, beispielsweise `input`, `x20`, `temperature`, `pressure` oder `valve`, werden abonniert. Die Subscription ist im aktuellen Programm mit einem Intervall von 100 ms angelegt.

3.2 Übertragung zum Browser

Wird eine Wertänderung gemeldet, speichert das Backend den aktuellen Wert. Zusätzlich wird für Werte mit dem Zustand 1 die aktive Zeit in Millisekunden aufsummiert. Etwa alle 200 ms sendet das Backend folgende Datenstruktur über WebSocket an die verbundenen Browser:

Listing 1: Vereinfachte WebSocket-Nachricht

```

{
  "channels": {
    "ns=2;i=2": 1,
    "ns=2;i=3": 22.7
  },
  "on_times": {
    "ns=2;i=2": 1540
  }
}

```

Die Weboberfläche aktualisiert ihre Widgets unmittelbar nach dem Empfang. Ein manuelles Neuladen der Seite ist nicht erforderlich.

3.3 Ablauf in acht Schritten

1. Ein Taster wird betätigt oder ein Sensorwert ändert sich.
2. Die Steuerung aktualisiert die zugehörige Prozessvariable.
3. Der OPC-UA-Server stellt den neuen Wert bereit.
4. Das FastAPI-Backend erhält die Änderung über seine OPC-UA-Subscription.
5. Das Backend aktualisiert Wert und Einschaltdauer.
6. Die Daten werden über WebSocket an den Browser übertragen.
7. Das React-Frontend aktualisiert das betroffene Widget.
8. Die Bedienperson sieht Status und Verlauf nahezu in Echtzeit.

4 Weboberfläche

4.1 Verbindungsstatus

Im Kopfbereich zeigt das Dashboard den Zustand der WebSocket-Verbindung an. Mögliche Anzeigen sind **Connected**, **Connecting**, **Disconnected** und **Error connecting**. Das Frontend versucht standardmässige die Backend-Ports 8000 und 9001.

4.2 Sensorauswahl

Über das Seitenmenü können verfügbare OPC-UA-Knoten als Widgets hinzugefügt oder entfernt werden. Die Auswahl sowie die eingetragene OPC-UA-Adresse werden im lokalen Speicher des Browsers gespeichert.

4.3 Widgets und Detailansicht

Ein Widget zeigt:

- den Namen beziehungsweise die Node-ID,
- den aktuellen numerischen Wert,
- den Zustand **ON** oder **OFF**,
- die aufsummierte **ON Time** in Millisekunden,
- optional Statistik und grafischen Verlauf.

Der Verlauf enthält die letzten 30 vom Browser empfangenen Werte. Für die Statusanzeige gilt ein Wert von 1 als **ON**; andere Werte werden als **OFF** behandelt. Analoge Messwerte werden dennoch numerisch im Widget angezeigt.

5 Inbetriebnahme

5.1 Voraussetzungen

Benötigt werden:

- Python 3 mit `fastapi`, `uvicorn`, `asyncua` und `pydantic`,
- Node.js und npm,
- ein OPC-UA-Server oder der mitgelieferte Simulator,
- ein aktueller Webbrowser.

Alle folgenden Befehle werden im Ordner `opc-dashboard` ausgeführt. Für Simulator, Backend und Frontend wird jeweils ein eigenes Terminal verwendet.

5.2 Variante A: Betrieb mit Simulator

1. Python-Abhängigkeiten installieren:

```
python -m pip install fastapi uvicorn asyncua pydantic
```

2. OPC-UA-Simulator starten:

```
python opcua_simulator.py
```

Der Simulator verwendet standardmäßig `opc.tcp://127.0.0.1:4843`. Ist der Port belegt, wählt er automatisch einen freien Port. Die gewählte Adresse wird in `opcua_simulator_config.json` gespeichert und vom Backend gelesen.

3. Backend starten:

```
python -m uvicorn backend.main:app --reload --port 8000
```

4. Frontend installieren und starten:

```
npm install  
npm run dev
```

Anschließend wird die von Vite angezeigte Adresse, üblicherweise `http://localhost:5173`, im Browser geöffnet.

5.3 Variante B: Verbindung mit einer realen Steuerung

Zuerst werden Backend und Frontend wie oben beschrieben gestartet. Danach wird im Seitenmenü die OPC-UA-Adresse der Steuerung eingetragen, zum Beispiel:

```
opc.tcp://192.168.1.61:4840
```

Mit **Connect** wird das Backend auf die neue Adresse umgestellt. Rechner und Steuerung müssen sich gegenseitig über das Netzwerk erreichen können. Firewalls müssen die verwendeten Ports zulassen.

6 Fehlerbehebung

Problem		Mögliche Ursache und Lösung
Dashboard zeigt Disconnected	Keine Sensoren sichtbar	Backend läuft nicht oder Port 8000/9001 ist blockiert. Backend starten und Firewall prüfen.
Werte ändern sich nicht		OPC-UA-Server ist nicht erreichbar, die Adresse ist falsch oder Knotennamen passen nicht zum Filter des Backends.
Connect funktioniert nicht		Verbindung zur Steuerung und Aktualisierung der Prozessvariablen prüfen. Beim Test mit Simulator dessen Terminalausgabe kontrollieren.
Analoger Wert zeigt OFF		Das Frontend sendet die Umschaltung derzeit an Port 8000. Das Backend sollte daher auf Port 8000 laufen.
		Die ON/OFF-Anzeige ist für digitale Werte ausgelegt. Der tatsächliche analoge Wert wird weiterhin numerisch angezeigt.

7 Grenzen und Sicherheit

Die aktuelle Implementierung ist ein Prototyp für Entwicklung, Unterricht und ein kontrolliertes lokales Netzwerk. Vor einem produktiven Einsatz sind insbesondere folgende Punkte zu beachten:

- Das Backend erlaubt derzeit CORS-Zugriffe von allen Ursprüngen.
- Im Programm ist keine Anmeldung für Dashboard oder WebSocket vorgesehen.
- Die OPC-UA-Verbindung verwendet in der aktuellen Form keine konfigurierte Zertifikatsprüfung.
- Die Verlaufsdaten werden nur im Arbeitsspeicher des Browsers gehalten und nicht dauerhaft gespeichert.
- Bei einem Neustart des Backends gehen die aufsummierten Einschaltdauern verloren.
- Für Zugriff außerhalb des lokalen Netzes sind Authentifizierung, TLS und restriktive Firewall-Regeln erforderlich.

8 Zusammenfassung

Das OPC-UA-Live-Dashboard macht technische Prozesswerte einfach im Browser sichtbar. Die Steuerung oder der Simulator stellt Variablen über OPC UA bereit. Das FastAPI-Backend abonniert diese Werte und überträgt sie per WebSocket an die React-Oberfläche. Dort können aktuelle Werte, digitale Zustände, Einschaltdauer und ein kurzer Verlauf beobachtet werden.

Die Trennung zwischen OPC-UA-Kommunikation und Weboberfläche ist dabei wesentlich: Das Backend übernimmt die industrielle Kommunikation, während der Browser ausschliesslich die aufbereiteten Daten darstellt. Dadurch bleibt das System verständlich, testbar und später erweiterbar.